

UNIVERSIDAD SIMÓN BOLÍVAR

Departamento de Computación y Tecnología de la Información

CI-3391 – Taller de Bases de Datos I

Trimestre Abril - Julio 2026

Informe técnico de la API RESTful de videovigilancia inteligente

Corte 4

Integrantes:

Carlos Moreno	21-10409
Daniel Quijada	20-10518
Daniela Gragirena	19-10543

1 Introducción

Se implementa la base de datos construida en fases anteriores con una API funcional. El objetivo es aplicar consultas SQL directas, funciones almacenadas, validación de datos y diseño de servicios HTTP sobre PostgreSQL, sin utilizar un ORM. El resultado permite que una aplicación cliente registre información operativa y consulte métricas o coincidencias sin acceder directamente al DBMS.

Este informe explica la implementación existente en el repositorio, la lógica interna de cada endpoint y las decisiones que organizan el sistema. La revisión comprende los archivos Python de la aplicación, los esquemas Pydantic, el modelo SQL, las funciones PL/pgSQL, el script de carga inicial y el archivo de dependencias.

La implementación cubre los entregables funcionales obligatorios y una de las opciones de puntos extra.

Además de lo exigido, el sistema incluye `GET /ubicaciones/{id}`, consultas de cámaras por ubicación y una ruta adicional para alertas activas.

2 Arquitectura de la solución

La aplicación adopta una arquitectura por capas sencilla. Los routers concentran la interfaz HTTP, los modelos Pydantic describen los contratos de entrada y salida, `database.py` administra la conexión y PostgreSQL conserva los datos y la lógica analítica.

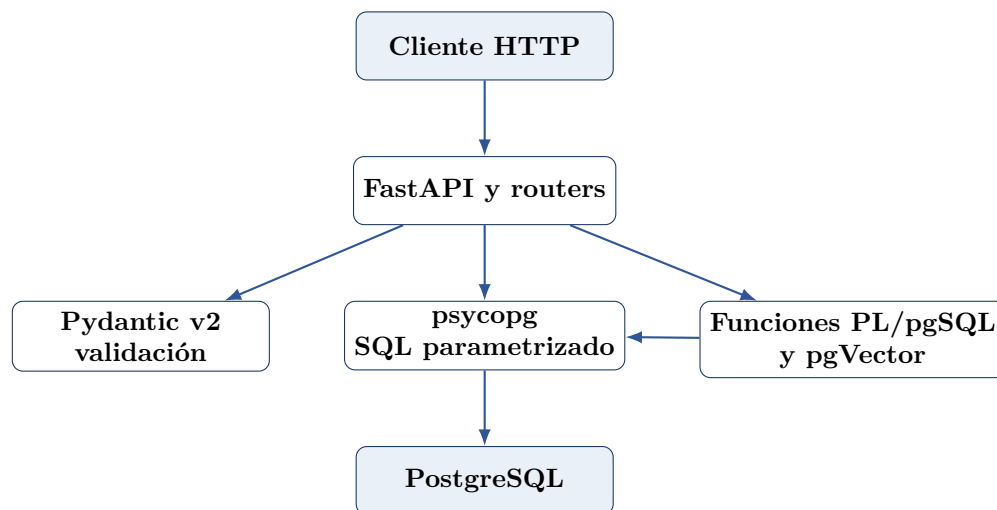


Figura 1: Flujo general de una solicitud

2.1 Flujo de una solicitud

- El cliente envía una solicitud HTTP a una ruta registrada en `main.py`.
- FastAPI interpreta los parámetros y solicita a Pydantic la validación del cuerpo cuando existe.
- La dependencia `get_db_connection` abre una conexión psycopg configurada con `dict_row`.
- El router ejecuta SQL parametrizado o llama a una función PL/pgSQL.
- La transacción se confirma en las operaciones de escritura y la conexión se cierra al finalizar.
- FastAPI serializa las filas devueltas como JSON y aplica el `response_model` cuando está definido.

2.2 Organización del código

Elemento	Responsabilidad
app/main.py	Crea la aplicación y registra todos los routers.
app/database.py	Abre conexiones psycopg con filas tipo diccionario.
app/routers	Contiene CRUD, analítica, vectores y funciones adicionales.
app/schemas	Define modelos Pydantic de entrada y salida.
schema.sql	Crea tipos ENUM, tablas, restricciones e índices relacionales.
functions.sql	Implementa las tres funciones analíticas reutilizadas por la API.
seed.sql y seed_100.csv	Cargan cien registros mediante una tabla temporal.
requirements.txt	Declara las dependencias de ejecución.

3 Conexión con PostgreSQL

`get_db_connection` crea una conexión psycopg por solicitud y configura `row_factory=dict_row`. Este formato convierte cada fila en un diccionario cuyas claves coinciden con los alias o nombres de columnas; FastAPI puede serializarlo directamente y Pydantic puede verificar la estructura de respuesta. El bloque `finally` garantiza el cierre de la conexión.

```
conn = psycopg.connect(DATABASE_URL, row_factory=dict_row)
try:
    yield conn
finally:
    conn.close()
```

Las consultas utilizan marcadores `%s` y tuplas de parámetros para evitar construir SQL con valores concatenados y reducir el riesgo de inyección. En `POST` y `PUT` se llama `conn.commit()` después de obtener la fila con `RETURNING`. Las consultas de lectura no requieren confirmación explícita.

4 Endpoints de ubicaciones

Método	Ruta	Lógica	Resultado
GET	/ubicaciones	Selecciona todas las filas.	200 y lista.
GET	/ubicaciones/{id}	Filtra por UUID; devuelve 404 si no existe.	200 o 404.
POST	/ubicaciones	Inserta y retorna la fila con UUID generado.	201.
PUT	/ubicaciones/{id}	Reemplaza los campos editables y usa <code>RETURNING</code> .	200 o 404.
DELETE	/ubicaciones/{id}	Elimina por UUID y comprueba <code>RETURNING</code> .	204 o 404.

4.1 Consulta y creación

GET /ubicaciones devuelve id, name, floor_no, zone_type, latitude y longitude. La ruta adicional por id recibe un UUID, ejecuta SELECT con filtro y produce 404 cuando fetchone() no retorna una fila. POST valida el cuerpo con LocationCreate; PostgreSQL genera el UUID mediante gen_random_uuid() y RETURNING entrega el registro creado.

4.2 Actualización y eliminación

PUT aplica una actualización completa de los cinco campos editables. La ruta no funciona como PATCH: el cliente debe enviar todos los valores exigidos por LocationCreate. DELETE retorna 204 sin cuerpo cuando encuentra la ubicación. Si existen cámaras vinculadas, la restricción foránea protege la integridad.

5 Endpoints de cámaras

Método	Ruta	Lógica	Resultado
GET	/camaras	Selecciona todas las cámaras.	200 y lista.
GET	/camaras/{id}	Busca por identificador de 12 caracteres.	200 o 404.
POST	/camaras	Inserta modelo, estado, visión nocturna y ubicación.	201.
PUT	/camaras/{id}	Actualiza atributos; conserva el id de la ruta.	200 o 404.

6 Endpoints de eventos

Método	Ruta	Lógica	Resultado
GET	/eventos	Devuelve todos los eventos y sus cuadros delimitadores.	200 y lista.
GET	/eventos/{id}	Busca un BIGINT y responde 404 si no existe.	200 o 404.
POST	/eventos	Inserta un evento asociado a una cámara.	201.

7 Endpoints analíticos

7.1 Tráfico por cámara

```
GET /analytics/cameras/{camera_id}/traffic?from=YYYY-MM-DD&to=YYYY-MM-DD
```

FastAPI convierte from y to al tipo date. El router llama get_camera_traffic(camera_id, from_date, to_date) y devuelve camera_id, hour_of_day y total_detections. La función filtra la cámara, incluye el día inicial y todo el día final mediante la condición menor que p_end + 1, agrupa por la hora extraída del TIMESTAMPTZ y ordena de 0 a 23. Solo aparecen las horas que tienen detecciones.

7.2 Resumen por zona

```
GET /analytics/zones/{zone_type}
```

El router llama `get_zone_summary(zone_type)`. Por cada `LOCATION` cuyo `zone_type` coincide, la función calcula cámaras con estado activo, total de eventos y alertas con severidad crítica. El resultado se ordena por nombre de ubicación. Cuando la función no devuelve filas, la API responde 404.

7.3 Resumen de alertas

```
GET /analytics/alerts/summary?days=30
```

`days` es un entero mayor o igual que uno y vale 30 por defecto. La consulta une `ALERT`, `EVENT` y `CAMERA`, limita los eventos al intervalo actual menos `days`, agrupa por severidad y cámara y ordena por severidad y cantidad descendente.

8 Búsqueda vectorial

`pgVector` almacena un embedding de 512 dimensiones en cada objeto. El operador `<=>` calcula distancia coseno: un valor menor indica mayor similitud. El sistema ofrece una búsqueda basada en un objeto registrado y otra basada en un vector enviado por el cliente.

8.1 Similitud desde un objeto existente

```
GET /objects/{object_id}/similar?threshold=0.20&limit=5
```

El router valida `object_id` como UUID, `threshold` entre 0 y 1 y `limit` entre 1 y 100. Primero comprueba que el objeto exista y tenga embedding; si no se cumple, responde 404. Luego ejecuta `find_similar_objects`. La función recupera el vector y el tipo del objeto de referencia, excluye el mismo UUID, limita la comparación a objetos del mismo tipo, conserva distancias menores o iguales al umbral y devuelve el contexto de cámara y fecha.

El JOIN con `IDENTIFIES` y `EVENT` puede devolver el mismo objeto más de una vez cuando fue identificado en varios eventos.

8.2 Similitud desde un embedding externo

```
POST /search/similar
```

`VectorSearchRequest` exige 512 números, un tipo opcional `persona` o `vehiculo` y un límite de 1 a 100. El router convierte la lista al literal aceptado por `pgVector`, descarta objetos sin embedding, aplica el filtro de tipo solo cuando se recibió y ordena por `embedding <=> vector`. `VectorSearchResult` retorna `id`, `type`, `color` y `distance`.

```
{
  "vector": [512 valores numericos],
  "tipo": "persona",
  "limit": 10
}
```

9 Importación de archivos CSV

```
POST /csv con multipart/form-data
```

La funcionalidad opcional procesa un archivo CSV en UTF-8 y exige como mínimo las columnas `ubicacion_nombre`, `camara_nombre`, `evento_id`, `evento_marca_tiempo`, `evento_confianza` y `objeto_tipo`. Cada fila ejecuta un flujo de upsert para `LOCATION`, `CAMERA` y `EVENT`, localiza o crea `OBJECT`, inserta `IDENTIFIES` sin duplicar la clave y crea `ALERT` cuando existe severidad.

La importación utiliza una transacción exterior y una transacción anidada por fila. Si una fila falla, su subtransacción se revierte y el procesamiento continúa. `added` y `updated` clasifican la fila según la existencia previa del evento; no representan el número total de entidades insertadas o modificadas.

Campo	Significado
<code>added</code>	Filas cuyo evento no existía antes del procesamiento.
<code>updated</code>	Filas cuyo evento ya existía y fue actualizado.
<code>rejected</code>	Filas revertidas por validación o error de base de datos.
<code>errors</code>	Detalle del número de fila y mensaje de error.

10 Funcionalidades adicionales

10.1 Cámaras por ubicación

```
GET /extra/cameras-by-location/{location_id}
```

La consulta une `CAMERA` y `LOCATION` y devuelve `id`, `modelo`, `estado`, `visión nocturna` y `nombre` de ubicación para un `UUID` determinado. Si no existen cámaras, retorna una lista vacía.

10.2 Alertas activas

```
GET /extra/active-alerts
```

La ruta lista alertas recientes.

11 Manejo de errores y respuestas HTTP

Código	Uso
200	Consultas y actualizaciones exitosas.
201	Creación de ubicación, cámara o evento.
204	Eliminación exitosa de una ubicación.
404	Ubicación, cámara, evento, zona u objeto no encontrado según la ruta.
422	Cuerpo, UUID, fecha, rango o vector inválido detectado por FastAPI y Pydantic.
500	Posible resultado de restricciones SQL no interceptadas o de la consulta adicional incompatible.

Los casos 404 están tratados explícitamente en las consultas por id, el resumen de zona y la búsqueda desde un objeto.

12 Decisiones de diseño

Decisión	Justificación
SQL directo	Mantiene visibles las consultas y joins.
Routers por dominio	Separa ubicaciones, cámaras, eventos, analítica, vectores y extras.
Dependencia de conexión	Centraliza la apertura y el cierre sin repetir código en cada ruta.
<code>dict_row</code>	Convierte filas a diccionarios listos para validación y JSON.
Funciones almacenadas	Utiliza lógica analítica.
ENUM y CHECK	Aplica integridad si el dato llega por la API o por otro cliente SQL.
RETURNING	Evita una segunda consulta después de insertar o actualizar.
Transacción por fila CSV	Permite continuar la carga cuando una fila individual falla.